Topic-Aware Multilingual Representation & LASER3-CO

Technical Specification, Mathematical Formulation & Complete Codebase Architecture

Project: Project NLP 2 (Bengali–Hindi Pilot & Kangri Transfer) | Server: him2 (SLURM) | Date: November 2026

1. Executive Summary & Objective

The objective of this project is to build sentence representations for low-resource languages (specifically Kangri, with Bengali–Hindi as our validation pilot) where sentences on the same topic cluster tightly together in vector space, while parallel translations remain reciprocal nearest neighbors. The project initially evaluated two dual-encoder models (**LaBSE** and **Multilingual-E5**) trained with contrastive translation loss and supervised topic loss (SupCon). This extension integrates **LASER3-CO** (Tan et al., EACL 2023) into the same rigorous training, evaluation, bootstrap statistical testing, and Streamlit dashboard ecosystem.

2. Methodological & Mathematical Framework

2.1 Dual-Encoder Fine-Tuning (LaBSE & E5)

The baseline pilot uses bi-encoders fine-tuned with two loss terms:
• **Translation Loss:** In-batch contrastive alignment pulling Bengali sentences to their Hindi counterparts.
• **Topic Loss (SupCon - Khosla et al., 2020):** Pulls together all sentences in the batch sharing the same topic label (regardless of language) while repelling sentences of distinct topics.

2.2 LASER3-CO Contrastive Distillation (Tan et al., 2023)

LASER3 features modular single-language encoders mapping into a shared multilingual space defined by LASER2. **LASER3-CO** continues training a student encoder via contrastive learning against a permanently frozen teacher:
• **Teacher Encoder:** Hindi encoder (hin_Deva), weights permanently frozen.
• **Student Encoder:** Bengali encoder (ben_Beng), updated with autograd gradients.
• **InfoNCE Loss with Negative Queue:**

InfoNCE Queue Formulation:

$$L_{CO} = -\log \left[\exp(q \cdot k_+ / \tau) / \left(\exp(q \cdot k_+ / \tau) + \Sigma_j \exp(q \cdot k_j / \tau) \right) \right]$$

where q = student embedding of Bengali sentence, k_+ = teacher embedding of true Hindi translation, and negatives k_j come from a FIFO memory queue of size **4096** plus batch target translations. Temperature τ **= 0.05**. No momentum encoder is required (unlike MoCo) because teacher is frozen.

2.3 Evaluated Model Variants

Variant Name	Type	Description & Loss Formulation
laser	Untrained Baseline	Untouched official LASER3 encoders for Bengali and Hindi; establishes the zero-shot baseline.
laser_co	LASER3-CO	Standard queue contrastive distillation (Q=4096, batch=32, τ=0.05, lr=1e-5, shuffled batches).
laser_co_filter	LASER3-CO-Filter	Length-sorted batches (harder negatives) + cosine filter dropping negatives with $\cos(k_i, k_+) \geq 0.9$.
laser_co_topic	LASER3-CO + Topic	LASER3-CO joint objective adding Supervised Contrastive topic loss (weight 1.0) to cluster topics.

3. Complete Codebase & File Structure

The repository follows a clean, modular layout where data preprocessing, training, evaluation, dashboard visualization, and SLURM workload batch jobs are decoupled. Below is the file tree and detailed breakdown of responsibilities.

```
Project_NLP_2/
+-- configs/
|   \-- pilot_bn.yaml             # Config: paths, datasets, hyperparams, compare pairs
+-- src/
|   +-- laser/
|   |   +-- check_laser.py        # Inspects encoders & checks language support
|   |   \-- train_laser_co.py     # LASER3-CO contrastive training loop (autograd + queue)
|   +-- eval/
|   |   +-- evaluate.py           # Primary scoring: KNN topic acc, translation error
|   |   +-- evaluate_laser.py     # Evaluates baseline & trained student checkpoints
|   |   \-- aggregate.py          # Multi-seed pooling (42, 43, 44) & paired bootstrap
|   +-- train/
|   |   \-- train.py              # Bi-encoder training for LaBSE & Multilingual-E5
|   \-- utils/
|       \-- common.py             # Config loading, seeding, path resolution, JSON
+-- dashboard/
|   +-- app.py                    # Streamlit multi-page dashboard router & sidebar
|   +-- pilot/
|   |   +-- bengali_hindi.py      # Main pilot dashboard: results, comparisons, curves
|   |   \-- laser3_co.py          # Dedicated LASER3-CO dashboard & benchmarks
|   \-- weeks/                    # Weekly progress logs for Kangri annotation
+-- slurm/
|   +-- install_laser.sbatch      # Environment installer (fairseq, laser_encoders)
|   +-- check_laser.sbatch        # Verification script for GPU module architecture
|   +-- train_laser_co.sbatch     # Single LASER3-CO training job
|   +-- train_laser_seq.sbatch    # Runs all 9 LASER training runs in ONE job
|   +-- evaluate_laser.sbatch     # Single checkpoint evaluation
|   +-- evaluate_laser_all.sbatch # Scores baseline + all 9 student checkpoints
|   +-- aggregate.sbatch          # Aggregates multi-seed results & statistical tests
|   \-- dashboard.sbatch          # Launches Streamlit web server on GPU node
\-- outputs/
    +-- checkpoints/              # Saved student model weights (best_student.pt)
    +-- results/pilot_bn/eval/    # results.csv, seeds_summary.csv, seeds_compare.csv
    \-- logs/                     # SLURM stdout and stderr log files
```

3.1 Key File Responsibilities

File / Module	Location	Functionality & Key Implementation Details
pilot_bn.yaml	configs/	Defines datasets (MASSIVE, SIB-200, IN22), hyperparameters (batch size, queue size 4096, temp 0.05, lr 1e-5), evaluation pairs for paired testing, and aggregate runs.
train_laser_co.py	src/laser/	Implements LaserSide (autograd forward pass on Transformer encoder), FIFO negative Queue, pre-filtering (σ=0.9), and checkpoint selection on dev.
evaluate_laser.py	src/eval/	Loads baseline or trained student checkpoint into Bengali encoder, executes evaluation over test sets using score_set, and appends to results.csv.
aggregate.py	src/eval/	Aggregates 3 repeat seeds (42, 43, 44), calculates mean/sd/min/max, and conducts group-paired bootstrap testing (10,000 resamples) for statistical significance.
app.py	dashboard/	Main Streamlit application router. Offers dedicated sidebar navigation to switch between general **Bengali–Hindi** pilot and dedicated **LASER3-CO** section.
laser3_co.py	dashboard/pilot/	Full 7-tab LASER dashboard: (1) Setup, (2) Results vs LaBSE/E5, (3) Statistical Significance, (4) Error Analysis, (5) Training Curves, (6) 2D Vector Space (t-SNE maps & star query), (7) Try It Live (interactive encoding, SentencePiece tokens, 1024-D vector, & 5-NN voting).
laser_utils.py	dashboard/pilot/	Shared pipeline utilities: cached GPU loader, PyTorch 2.6+ unpickler compatibility, student checkpoint injection, SentencePiece tokenization, and language detection.

4. Evaluation Metrics & Verification Protocol

All models across LaBSE, E5, and LASER families are scored using the exact same evaluation engine (src/eval/evaluate.py:score_set) to ensure strict, unbiased comparability.

• **1. Topic Accuracy (Bengali side, %):** 5 nearest neighbor sentences vote on the topic label. For IN22-Conv, sentences from the same conversation are masked out.

• **2. Translation Error (Bengali→Hindi, %):** Percentage of Bengali sentences whose closest Hindi sentence (by cosine similarity) is NOT its true translation. Lower is better.

• **3. Topic Accuracy across Languages (%):** Bengali test sentences queried against Hindi reference sentences.

• **4. Cluster Agreement (NMI / Normalized Mutual Information):** Unsupervised k-means alignment with gold labels.

• **5. 95% Bootstrap Confidence Intervals:** 1,000 resamples over test sentences; paired bootstrap differences across 3 repeat training seeds (42, 43, 44) to guarantee robustness.

5. Step-by-Step Execution Guide on SLURM (Server him2)

All operations run strictly on GPU compute nodes via sbatch from /raid/home/loitongbam/Project_NLP_2.

Step	SLURM Command	Purpose & Expected Outcome
Step 1	sbatch slurm/check_laser.sbatch	Inspects Bengali & Hindi encoders, downloads model files, confirms PyTorch 2.6+ unpickling compatibility.
Step 2	sbatch slurm/evaluate_laser.sbatch configs/pilot_bn.yaml	Evaluates the untouched LASER baseline across SIB-200, IN22-Gen, and IN22-Conv. Populates results.csv.
Step 3	sbatch slurm/train_laser_co.sbatch configs/pilot_bn.yaml co 42 --max_steps 20	Runs 20-step smoke test; verifies loss decay, autograd propagation, and checkpoint saving in outputs/checkpoints/.
Step 4	sbatch slurm/train_laser_seq.sbatch configs/pilot_bn.yaml \ co:42 co:43 co:44 \ co_filter:42 co_filter:43 co_filter:44 \ co_topic:42 co_topic:43 co_topic:44	Executes all 9 full training runs sequentially in ONE batch job to respect cluster user job limits.
Step 5	sbatch slurm/evaluate_laser_all.sbatch configs/pilot_bn.yaml	Evaluates the baseline and all 9 trained student checkpoints. Logs metrics to results.csv and per-sentence TSVs.
Step 6	sbatch slurm/aggregate.sbatch configs/pilot_bn.yaml	Combines repeat seeds, executes paired bootstrap tests (LASER vs variants), writes seeds_compare.csv.
Step 7	sbatch slurm/dashboard.sbatch [port]	Launches Streamlit dashboard on compute node. Access via SSH tunnel to inspect live results, curves, and error analysis.

6. Key Findings Expected & Comparison with Benchmarks

• **Translation Error:** LASER3-CO is specifically optimized for translation matching. A significant drop in translation error (e.g., from ~10% down to <2% on SIB-200) is expected for laser_co and laser_co_filter.
• **Topic Accuracy:** Pure contrastive translation learning is not expected to significantly move topic accuracy. Only the laser_co_topic variant incorporates supervised topic clustering loss (SupCon) and is expected to increase topic accuracy.
• **Benchmark Comparison:** When viewing the final table in the dashboard, LASER models appear directly alongside LaBSE and Multilingual-E5, providing an end-to-end comparison of all three encoder architectures under identical test conditions.